# nwergm: Exponential-family random graph models in Stata

Thomas U. Grund
Institute of Sociology, RWTH Aachen University, Germany

**Abstract.** Exponential-family random graph models (ERGMs) are a standard tool for the statistical analysis of network data, but Stata users have had to fit them with other software. `nwergm` fits ERGMs in Stata. It uses the same estimation algorithms, control defaults and terms as the `ergm` package of the statnet project, entirely within Stata, and implements 123 of the 139 terms of `ergm` 4.12.0 fully and two in part. It handles directed, undirected and two-mode networks, curved and operator terms, and sampling and observation constraints; it estimates by maximum pseudolikelihood, Monte Carlo maximum likelihood, stochastic approximation or contrastive divergence, and offers Monte Carlo diagnostics, goodness of fit, information criteria, prediction and simulation. The sampler and the change statistics run in a compiled plugin. On the examples of the article the estimates agree with those of `ergm` to within 0.09 standard errors; on eight benchmark models a fit is faster by a factor of 2.1 to 4.1. The article describes the syntax, the term library, the estimation and the diagnostics, with the Florentine marriages, an advice network among lawyers and a two-mode network as worked examples.

**Keywords:** st0003, nwergm, exponential-family random graph models, ERGM, network statistics, Monte Carlo maximum likelihood, Stata plugins, nwcommands

## 1 Introduction

An exponential-family random graph model (ERGM) treats a network's ties as the joint outcome of a small set of structural and covariate-driven tendencies, such as density, reciprocity, homophily, triadic closure and degree heterogeneity, which are estimated together; Lusher et al. (2013) give the standard treatment of the theory, estimation, and interpretation involved. The `ergm` package of the statnet project (Handcock et al., 2008; Hunter et al., 2008b; Krivitsky et al., 2023) has been the standard implementation for close to two decades. A Stata user who wanted to fit an ERGM had to leave Stata, export the network, fit the model in other software, such as `ergm` in R or PNet (Wang et al., 2009), and import the results back.

`nwergm` fits ERGMs in Stata. It is the ERGM command of `nwcommands` (Grund, 2026a), a family of Stata packages for network analysis, and it fits models to a network that has been declared with `nwset` or loaded with one of the `nwcommands` import commands. It implements nearly all terms of `ergm` 4.12.0 (Table 1), estimation by maximum pseudolikelihood, Monte Carlo maximum likelihood, stochastic approximation and contrastive divergence, the sampling and observation constraints, the treatment of missing dyads, and the postestimation tools `mcmc.diagnostics()`, `gof()`, `simulate()`, `logLik()` and `predict()`, each under a Stata name. Where this article says that `nwergm` does something as `ergm` does, it means `ergm` 4.12.0 and the terms and options that `nwergm` implements.

**Provenance.** nwergm was developed from ergm: the term definitions and change statistics, the pseudolikelihood and Monte Carlo estimators, the control defaults and the diagnostics follow the R package and its published description (Hunter and Handcock, 2006; Hunter, 2007; Morris et al., 2008; Hummel et al., 2012; Geyer and Thompson, 1992). nwergm is not affiliated with or endorsed by the statnet project, and errors are mine. Where ergm and nwergm differ, Section 10 says so.

# 2 Installation

nwergm is a package of its own. It uses the network objects and data handling of nwcommands (Grund, 2026a), which must be installed first; installation is repeated here in full so this article stands on its own. Stata 17 or later is needed (the software is tested with Stata 17, 18 and 19; StataBE is enough). From within Stata:

```
. net install nwcommands, from(https://nwcommands.org/net/) replace
. nwinstall, everything
```

The first command installs nwinstall. The second installs the nwcommands core (the full command set, the help files and the dialog boxes), the four model packages including nwergm, and a "Network Analysis" menu in Stata, in one step, and prints the installed versions at the end. Only the core and nwergm are needed for this article: nwinstall, all followed by nwinstall, modelpkg(nwergm) installs just those two (add nogui to skip the menu). Repeating the lines updates the installation. nwergm has its own repository and version, its own Mata library and the compiled plugin for the platform (macOS, Linux or Windows); no compiler is needed. help nwergm opens the documentation, and the dialog db nwergm sets every term and option. The manual of nwergm (Grund, 2026c) is the technical reference: it documents every option and term, and it is archived at Zenodo under the DOI of all versions, https://doi.org/10.5281/zenodo.23661331. The source code of all packages is public at https://github.com/thomas-u-grund, with nwergm at https://github.com/thomas-u-grund/nwergm.

## Versions

The logs in this article were produced with nwergm 1.5.6 (9 November 2026) and nwcommands 3.17.2 in Stata 18; the comparisons and timings of sections 9 and 9.3 were made with nwergm 1.5.4, whose estimation code is the same. nwergm needs nwcommands version 3.6.0 or later. Releases follow the usual major.minor.patch scheme: a change in the first number may change results or syntax, the second adds features, and the third fixes errors. The website explains how to install these releases instead of the current ones: https://nwcommands.org/nwcommands/install/. nwergm version reports the installed release, the version of the nwcommands core, the plugin in use and the version of ergm that the results were compared with; e(version) stores the release after each fit, so a published analysis can state which release produced it. nwversion lists the core and all installed model packages. The do-file that produces every log of this article, gen_logs.do, is part of the supplemental materials.

## License

`nwergm` is free software under the GNU General Public License, version 3, as are the `nwcommands` core and the other model packages. `nwergm` is derived from `ergm` (Handcock et al., 2008), which is distributed under the same license. Further worked tutorials are at `https://nwcommands.org/nwergm/`.

# 3 ERGM framework

An ERGM models an entire network $y$ as one draw from an exponential-family distribution over the set $\mathcal{Y}$ of all possible networks on a fixed set of nodes:

$$P_\theta(Y = y) = \frac{\exp(\theta^\top g(y))}{\kappa(\theta)}, \qquad y \in \mathcal{Y}, \tag{1}$$

where $g(y)$ is a vector of network statistics (density, reciprocity, triadic closure and so on), $\theta$ is the corresponding vector of parameters, and $\kappa(\theta) = \sum_{y' \in \mathcal{Y}} \exp(\theta^\top g(y'))$ normalizes the distribution over every network on the same node set: a sum with $2^{n(n-1)}$ terms for a directed network on $n$ nodes, intractable at any realistic size. In a curved model, $\theta$ is a function of a smaller parameter vector, as for the geometrically weighted terms whose decay is estimated (Hunter and Handcock, 2006).

Every term enters the model through its *change statistic*: how much the term's contribution to $g(y)$ changes if a single tie $y_{ij}$ is toggled, holding every other tie fixed,

$$\Delta g_{ij}(y) = g(y^+_{ij}) - g(y^-_{ij}), \tag{2}$$

where $y^+_{ij}$ and $y^-_{ij}$ denote $y$ with the $ij$ tie forced present and absent. The conditional log odds of a single tie reduce to a linear form in the change statistic regardless of how complicated $g(y)$ is:

$$\log \frac{P_\theta(Y_{ij} = 1 \mid Y_{-ij} = y_{-ij})}{P_\theta(Y_{ij} = 0 \mid Y_{-ij} = y_{-ij})} = \theta^\top \Delta g_{ij}(y). \tag{3}$$

So a coefficient is the change in the conditional log odds of a tie per unit of the statistic, given the rest of the network. The pseudolikelihood uses this form: it is the likelihood of a logistic regression of the ties on their change statistics, which play the role of the covariates of dyad $ij$.

Whether the pseudolikelihood is also the likelihood depends on whether the terms are dyad-independent or dyad-dependent. If every term is dyad-independent (its change statistic depends only on nodal attributes and dyadic covariates of $i$ and $j$, and not on other ties), the joint likelihood factors into independent per-dyad logistic terms, and the maximum pseudolikelihood estimate *is* the maximum likelihood estimate (van Duijn et al., 2009). Dyad-dependent terms, such as `mutual`, the geometrically weighted and shared-partner terms, the degree-distribution and triad terms, break the factorization, and $\kappa(\theta)$ cannot be avoided. `nwergm` then maximizes the likelihood by Monte Carlo: it simulates networks from a trial $\theta$ by a Metropolis–Hastings sampler and updates $\theta$ until the simulated statistics match the observed ones on average (Geyer and Thompson, 1992; Hummel et al., 2012). The log-likelihood at the estimate, needed for the information criteria, is evaluated by bridge sampling from the dyad-independent submodel.

# 4 The nwergm command

## 4.1 Basic syntax

. `nwergm` [*netname*] `,` `edges` [*terms*] [`structural(`*termlist*`)`] [`operators(`*oplist*`)`] [*estimation options*]
. `nwergm simulate` [*nodes*] `,` [*terms* `theta(`*numlist*`)`] [*options*]
. `estat gof` | `estat mcmcdiag` | `estat ic`

A model is specified as a list of term options, the way a Stata regression command's option list specifies covariates. `edges` is required; *netname* defaults to the current network, so a model is fitted directly to whatever `nwset` or `nwwebuse` last produced. Node attributes are numeric variables of the current dataset (a string attribute is first converted with `encode`); dyadic covariates are networks in memory. Coefficients carry the names of `ergm`'s coefficients with the dot written as an underscore, such as `nodematch_seat`. An estimation result is an ordinary Stata estimation result: `e(b)`, `e(V)`, `estimates store`, `lrtest`, `test` and `predict` work as usual.

**Network types.** `nwergm` fits directed, undirected and two-mode (bipartite) networks. A directed two-mode network is refused, as are temporal networks. A network with tie values is fitted as the binary graph in which a tie is present when its value is not zero, as `ergm` does without its option `response=`; `nwergm` prints one note that the values are ignored (and no note when every value is 1). The terms for valued networks of `ergm` are not available (Section 10).

## 4.2 Term library

Table 1 lists the terms of `ergm` 4.12.0 by family and the number that `nwergm` implements. Of the 139 terms, 123 are implemented as in `ergm`, two in part, and 14 are not implemented (13 terms for valued networks and the operator `B`). Apart from these, the library is complete. Thirty-three of the terms apply to two-mode networks only. The definitions and the interpretation of the terms are those of `ergm` (Morris et al., 2008); Lusher et al. (2013) discuss when to prefer one over another. Figure 1 shows what six terms of the examples count.

**How the terms are written.** The most common terms have options of their own: `mutual`, `nodematch()`, `nodecov()`, `nodefactor()`, `nodemix()`, `edgecov()`, `degree()`, `kstar()`, `triangle`, `gwesp()`, `esp()` and many more; the table gives the spelling. Terms whose R name contains a mode digit followed by a letter keep the name with the digit moved to the end, because Stata does not allow a digit followed by a letter in an option name: `b1degree` is `bdegree1()`, `gwb2degree` is `bgwdegree2()`. The less common structural terms of `ergm` (`twopath`, `isolates`, `cycle`, `degree` with `by=`, `ttriple`, `simmelian`, `triadcensus`, and others) are written in R's notation inside `structural()`, with R's argument names and defaults (a vector such as R's `c(1,2)` is written `1 2`): `structural(isolates cycle(k=3 4) degree(d=1 2 by=sex))`. The terms of the form `gwesp(`$\alpha$`)` have a fixed decay $\alpha$; `gwespfree()`, `gwdegreefree()`, `gwdspfree()`, `gwnspfree()`, `gwodegreefree()` and `gwidegreefree()` estimate the de-

Table 1: The terms of `ergm` 4.12.0 by family, and how many `nwergm` implements.

| Family | `ergm` | `nwergm` | Examples (`nwergm` syntax) |
|---|---|---|---|
| Density, reciprocity | 5 | 5 | `edges`, `mutual`, `hamming()`, `structural(density meandeg)` |
| Degrees, stars, concurrency | 35 | 35 | `degree()`, `odegree()`, `idegree()`, `kstar()`, `degrange()`, `concurrent`, `structural(isolates twopath degree1.5)`, two-mode `bdegree1()`, `bstar2()` |
| Shared partners | 5 | 5 | `esp()`, `dsp()`, `structural(nsp())`, `bdsp1()`; `type()` on directed networks |
| Triads, cycles, paths | 18 | 18 | `triangle`, `ctriple`, `transitiveties`, `cyclicalties`, `structural(cycle() ttriple threetrail simmelian triadcensus balance)` |
| Curved (geometrically weighted, alternating) | 11 | 10 + 1 part | `gwesp()`, `gwdsp()`, `gwnsp()`, `gwdegree()`, `gwodegree()`, `gwidegree()`, two-mode `bgwdegree1()`, `bgwdsp1()`; estimated decay: `gwespfree()` and five more; `altkstar` |
| Node covariates | 34 | 34 | `nodecov()`, `nodefactor()`, `nodematch()`, `nodemix()`, `absdist()`, `sociality()`, `sender`, `receiver`, directed `nodeicov()`, `nodeofactor()`, two-mode `bcov1()`, `bfactor1()` |
| Dyadic covariates | 3 | 3 | `edgecov()`, `dyadcov()`, `attrcov()` |
| Operators | 15 | 13 + 1 part | `operators(Sum() Prod() S() F() NodematchFilter() Symmetrize() Project() I() For() Label() Log() Exp() Offset() Curve())` |
| Valued networks only | 13 | 0 | — |
| All | 139 | 123 + 2 part | 125 of 139 (89.9%) |

"part" means implemented in part: `altkstar` with an estimated decay (by pseudolikelihood only) and `Curve()` for linear maps (Section 10).

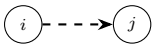
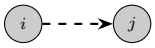
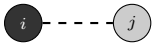
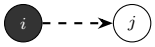
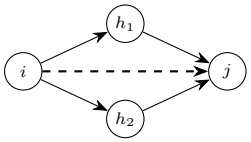
| Diagram | Statistic $g(y)$ | Description |
|---|---|---|
|  | $\sum_{i,j} y_{ij}$ | `edges`: the number of ties; the baseline tendency to form ties. |
|  | $\sum_{i<j} y_{ij} y_{ji}$ | `mutual`: $i$ names $j$ who already names $i$. |
|  | $\sum_{i,j} y_{ij} [v_i = v_j]$ | `nodematch(practicen)`: ties between lawyers in the same practice (here shaded alike). |
|  | $\sum_{i<j} y_{ij} (w_i + w_j)$ | `nodecov(wealth)`: the wealth of the two families of each marriage, added up. |
|  | $\sum_{i,j} y_{ij} [v_i = 2]$ | `nodeofactor(statusn)`: the ties sent by partners (level 2, dark). |
|  | $\sum_{i,j} y_{ij}\, e^{\alpha}\{1-(1-e^{-\alpha})^{p_{ij}}\}$ with $p_{ij} = \sum_h y_{ih} y_{hj}$ | `gwesp()`: the shared partners $h$ of the tie $i \to j$, with declining weights, so a tie with many shared partners adds little beyond the first few. |

Figure 1: What the terms count. The dashed tie is the one whose presence the term responds to; solid arrows are ties that already exist. The statistic is what the term adds up over the network ($y_{ij} = 1$ if $i$ names $j$, $v_i$ is a categorical attribute, $w_i$ a numeric one, and $\alpha$ is the decay parameter of `gwesp()`). A tie that completes the configuration is more likely when the coefficient is positive. The terms are those of the examples in Section 7; `nodecov(wealth)` is drawn undirected, as the Florentine marriages are.

cay together with the other coefficients (R's `fixed=FALSE`); the number in parentheses is then the starting value. On a directed network the shared-partner terms take `type(OTP|ITP|OSP|ISP|RTP)`, selecting one of the five definitions of a directed shared partner, the default being `OTP` as in R.

**Operators.** The option `operators()` applies terms to a derived network or combines them, written like R's operator terms: `Sum`, `Prod`, `S` (the subgraph induced by a set of nodes), `F` (the network without the ties that fail a filter), `NodematchFilter`, `Symmetrize`, `Project` (the one-mode projection of a two-mode network), `I`, `For`, `Label`, `Log`, `Exp`, `Offset`, and `Curve` for linear maps. For example, `operators(Symmetrize(triangle, rule(strong)))` counts triangles in the undirected network of reciprocated ties (Section 7.4). Operators cannot be combined with constraints, and a curved term cannot be placed inside an operator.

**Constraints, offsets and observation.** `constraints()` restricts the set of networks on which the model is defined, written as in R's `constraints=`: `edges` (the number of ties is fixed), `degrees`, `odegrees`, `idegrees`, `b1degrees`, `b2degrees`, `degreedist`, `bd()` (bounded degrees), `blocks()`, `blockdiag()`, `fixedas()`, `fixallbut()`, `observed`, `egocentric()` and `dyadnoise()`, joined by `+` or `-`. A statistic that a constraint holds constant is dropped from the estimation with a note, as in R. `obsconstraints()` describes how the network was observed (for example which dyads were not observed); `estat gof` and `nwergm simulate` use the constraints of the fit. The option `offset(`*coefname #*`)` holds a coefficient fixed at a known value.

**Missing data.** As in R, a missing dyad is not treated as a non-tie. Missing dyads leave the pseudolikelihood; for a dyad-dependent model a second, constrained sample in which only the missing dyads are free enters the estimating equations (Handcock and Gile, 2010). `e(N)` is the number of informative dyads and `e(nmissing)` the number of missing dyads. A missing value in a node attribute or a dyadic covariate is an error, as in R.

# 5 Estimation

**Choosing the method.** If every term is dyad-independent, maximum pseudolikelihood is the maximum likelihood estimate, and `nwergm` detects this automatically, reports the fit as an MPLE and runs no simulation. Any dyad-dependent term, a dyad-dependent constraint, a curved term or an observation constraint switches the default to Monte Carlo maximum likelihood, with the pseudolikelihood estimate as the starting value. The option `estimate(mle|mple|cd)` chooses the estimator, and `mainmethod(mcmle|sa)` chooses the Monte Carlo algorithm.

**Pseudolikelihood.** The pseudolikelihood is maximized by logistic regression on the matrix of change statistics. `mpletype(penalized)` gives the penalized (Firth, 1993) estimate, which is finite when the ordinary estimate does not exist; `mplecheck(on)` tests whether the estimate exists and warns as R does;

`mplecovariance(godambe|bootstrap)` replaces the covariance of a pseudolikelihood fit of a dyad-dependent model.

**Monte Carlo maximum likelihood.** The algorithm follows Hummel et al. (2012): each iteration simulates a sample at the current coefficients, takes a Newton step of adaptive length, and stops when a test on the simulated statistics (`mcmletermination()`: `confidence`, `hummel`, `hotelling` or `none`) says that the observed statistics lie inside the sampled distribution. The Metropolis–Hastings proposal is R's default, SPDyad: three quarters of the steps are the tie/no-tie steps of Morris et al. (2008), the others toggle a dyad drawn from the dyads whose nodes share a partner, which helps models with strong triadic effects; `mcmcprop(sparse)` gives tie/no-tie steps alone, and `proposal(uniform)` is also available. The controls have the names and defaults of R's `control.ergm()`, and the algorithm that uses them is R's, including its adaptive sample size. The first iteration starts with `mcmcinterval()` 1,024 proposals between recorded draws and a burn-in of 16 times the interval (`mcmcburnin()`, 16,384); an `mcmcinterval()` given without `mcmcburnin()` also sets the burn-in to 16 times its value. Each iteration records a base sample of $\max(256, 32p)$ draws for $p$ parameters (`mcmcsamplesize()`) and continues sampling until the effective sample size of the estimating equations reaches the target of 64 (`mcmleeffectivesize()`, R's `MCMLE.effectiveSize`); as in R, the effective size, the test of the burn-in and the covariance of the convergence test are computed from the spectral density at zero of a multivariate autoregression fitted to the estimating equations; the burn-in is estimated from the sample itself, the chain is thinned by two whenever the sample has doubled, and the next iteration starts with half the interval that the sample needed; the thinning interval that `estat mcmcdiag` reports is therefore that of the final sample, which can be larger than the interval requested. The default termination is R's test `confidence`: the fit stops when, with 99% confidence, the estimating equations at the updated coefficients lie inside a region of 0.1 of their covariance; a failed test raises the target effective size and the base sample size, by at most a factor of 2. The covariance matrix is taken from the sample of the last iteration, and `mcmleiterations()` is 60. The number of recorded draws is therefore not a fixed setting: `e(mcmc_samplesize)` gives the number in the final sample. `mcmleeffectivesize(0)` asks instead for fixed-size samples (3,000 draws in every iteration unless `mcmcsamplesize()` says otherwise); much shorter chains, an interval of 50 and a burn-in of 3,000, can be set with `mcmcburnin(3000) mcmcinterval(50)`, at the risk of poor mixing. The log-likelihood is evaluated by bridge sampling (`bridgesteps()`, 16 as in R; skipped with `nologlik`), with the sample size, interval and burn-in of the last iteration of the fit, as in R; it is not evaluated after a curved fit by Monte Carlo maximum likelihood.

**Contrastive divergence and stochastic approximation.** `estimate(cd)` is contrastive divergence (Krivitsky, 2017): short chains started from the observed network, used to find a starting point or a rough estimate; the covariance reported is that of the short-chain sample. `mainmethod(sa)` is the stochastic approximation algorithm of Snijders (2002) in the form of R's `main.method="Stochastic-Approximation"`. `init()` supplies starting values, one per coefficient, with a missing value for a coefficient to be

estimated.

**Reproducibility.** With `seed()` the same call gives the same numbers in any session, on the same platform and plugin build. A different operating system, a different plugin build or a different value of a control option generally changes the chain and, within Monte Carlo error, the estimates.

# 6 Postestimation

**estat mcmcdiag.** After a Monte Carlo fit, `estat mcmcdiag` reports diagnostics of the final simulated sample: a header with the size of the sample, the burn-in, the thinning interval and the acceptance rate, and for each statistic the mean, standard deviation, autocorrelation at the thinning interval, effective sample size and Geweke statistic (a note names the requested interval when the adaptive sampler raised it). Option `heidel` adds the Heidelberger–Welch test, and option `report` adds the report of R's `mcmc.diagnostics()` (sample statistics centered at the observed ones, the test that they differ from zero, cross- and autocorrelations). With `plot` it draws trace and density plots. A satisfied convergence test is necessary but not sufficient; the last note advises `mcmleeffectivesize()` (or `mcmcsamplesize()` and `mcmcinterval()` for a fixed-size sample).

**estat gof.** `estat gof` follows R's `gof()`: it simulates networks from the fitted model and, for each chosen statistic, shows the observed value, the minimum, mean and maximum over the simulations and a Monte Carlo $p$-value. The statistics are the model statistics and the distributions of degree, in-degree, out-degree, two-mode degrees, edgewise and dyadwise shared partners, geodesic distance and the triad census, chosen with `gof()`; the default is that of R for the type of network. `plot` draws box-and-whisker plots of the simulated distributions with the observed one overlaid, as R's `plot.gof()` (Hunter et al., 2008a).

**Fit statistics and prediction.** After every fit the log-likelihood, its Monte Carlo standard error, deviance, AIC and BIC are stored (BIC with $N$ the number of informative dyads); `estat ic` displays them, `estimates stats` and `lrtest` compare models. `predict, matrix()` and `predict, frame()` return the conditional tie probabilities or log odds of each dyad given the rest of the observed network, as `predict.ergm`.

**nwergm simulate.** `nwergm simulate` draws networks from a model. Without a number of nodes it draws from the last fit, starting from the fitted network and using the fitted coefficients and constraints, as R's `simulate()`; `coef()` and `basis()` change the coefficients and the starting network. With a number of nodes, the terms and `theta()` it draws from a fully specified model. `output(network)` creates ordinary networks, `output(stats)` returns the statistics of the draws and `output(edgelist)` an edge list. A fitted two-mode model gives two-mode networks.

**Stored results.** Besides `e(b)` and `e(V)`, `nwergm` stores the numbers of informative and missing dyads (`e(N)`, `e(nmissing)`), of nodes and ties, the log-likelihood and its relatives (`e(ll)`, `e(ll_0)`, `e(ll_se)`, `e(df_m)`, `e(aic)`, `e(bic)`, `e(deviance)`), and, after Monte Carlo estimation, `e(converged)`, `e(mcmle_iterations)`, the acceptance rate, the burn-in, interval and sample size used, `e(native)` (1 if the plugin ran), and the matrix `e(mcmcsample)` of the final sample of statistics. `e(version)` holds the package version; it should be quoted with an analysis.

# 7 Remarks and examples

Every fit below that uses the Monte Carlo sampler has a `seed()`.

## 7.1 A small network: the Florentine marriages

The marriage network among 16 Renaissance Florentine families (Padgett and Ansell, 1993) has 20 ties; `wealth` records each family's wealth in 1427, in thousands of lira. In network terms, the families are the nodes, and a marriage between two families is an undirected tie between them; the network can be drawn with `nwplot` (Grund, 2026b) (Figure 2).

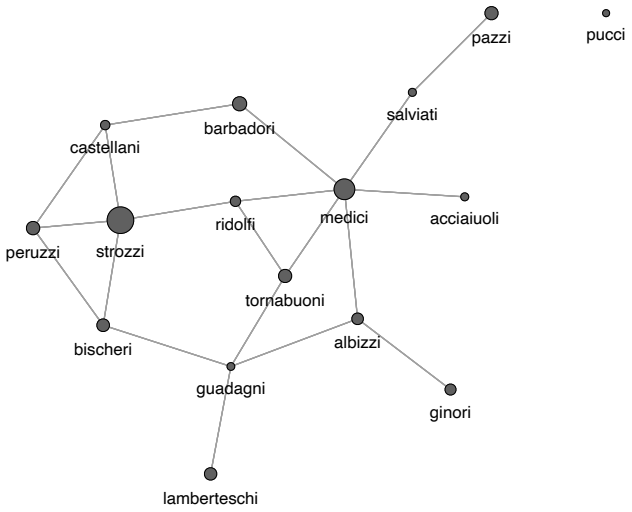



Figure 2: Marriage ties among 16 Florentine families (`nwplot flomarriage`, do-file `plot_networks.do`). Each marker is a family and each line a marriage. The size of a marker is proportional to the family's wealth. The position of a family on the page has no meaning of its own; the layout only places linked families near each other.

A family with many lines (the Medici have six) has many marriage partners; Pucci

has no marriage tie in these data. The wealthiest families, the Strozzi and the Medici, have the most marriage partners. Whether this is more than chance, given the number of ties, is a question for a model. An edges-only model is the ERGM analogue of an intercept-only regression:

```
. nwwebuse florentine, nwclear
. nwergm flomarriage, edges
Exponential-family random graph model
Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MPLE
```

| flomarriage | Coefficient | Std. err. | z | P>\|z\| | [95% conf. | interval] |
|---|---|---|---|---|---|---|
| edges | -1.609438 | .244949 | -6.57 | 0.000 | -2.089529 | -1.129347 |

```
Maximum pseudolikelihood estimate (not full ERGM maximum likelihood unless the m
> odel is dyad-independent).
             Null Deviance:     166.4  on 120 degrees of freedom
         Residual Deviance:     108.1  on 119 degrees of freedom
Log-likelihood:   -54.0673 (df = 1)   AIC:     110.1   BIC:     112.9   (MC std.
>  err. =       0)
```

The coefficient is the log odds of a tie, $\log(20/100) = -1.609$, and the fit is an MPLE that is also the maximum likelihood estimate. A term for wealth is again dyad-independent and keeps the estimation at the pseudolikelihood:

```
. nwergm flomarriage, edges nodecov(wealth)
Exponential-family random graph model
Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MPLE
```

| flomarriage | Coefficient | Std. err. | z | P>\|z\| | [95% conf. | interval] |
|---|---|---|---|---|---|---|
| edges | -2.594929 | .5360561 | -4.84 | 0.000 | -3.64558 | -1.544278 |
| nodecov_wea~h | .0105459 | .0046743 | 2.26 | 0.024 | .0013845 | .0197073 |

```
Maximum pseudolikelihood estimate (not full ERGM maximum likelihood unless the m
> odel is dyad-independent).
             Null Deviance:     166.4  on 120 degrees of freedom
         Residual Deviance:     103.1  on 118 degrees of freedom
Log-likelihood:   -51.5543 (df = 2)   AIC:     107.1   BIC:     112.7   (MC std.
>  err. =       0)
```

The statistic of `nodecov(wealth)` adds up the wealth of the two families of every marriage. The log odds of a marriage between two families therefore rise by 0.011 for every thousand lira that the two have together: wealthier families have more marriage ties ($z = 2.26$).

Adding `gwesp()`, which models triadic closure and is dyad-dependent, moves the estimation to Monte Carlo maximum likelihood:

```
. nwergm flomarriage, edges nodecov(wealth) gwesp(.5) seed(2026)
Exponential-family random graph model
Network:               =  flomarriage
Nodes:                 =  16
Ties:                  =  20
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  486
```

| flomarriage | Coefficient | Std. err. | z | P>\|z\| | [95% conf. | interval] |
|---|---|---|---|---|---|---|
| edges | -2.601579 | .5694501 | -4.57 | 0.000 | -3.71768 | -1.485477 |
| nodecov_wea~h | .010939 | .0050513 | 2.17 | 0.030 | .0010387 | .0208393 |
| gwesp_.5 | -.0180976 | .2538451 | -0.07 | 0.943 | -.5156249 | .4794296 |

```
MCMLE converged after 1 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...
            Null Deviance:     166.4  on 120 degrees of freedom
        Residual Deviance:     103.1  on 117 degrees of freedom
Log-likelihood:   -51.5598 (df = 3)   AIC:     109.1   BIC:     117.5   (MC std.
>  err. =   .0031)
```

The wealth effect stays (0.011, $z = 2.17$), while the closure coefficient is close to zero: these marriages show no tendency toward closed triangles beyond what wealth and the number of ties explain. The estimators give similar answers for one model with a triangle term (the pseudolikelihood estimate is not the maximum likelihood estimate once the model is dyad-dependent); the contrastive divergence estimate is the roughest, with larger standard errors:

```
. quietly nwergm flomarriage, edges nodecov(wealth) triangle seed(2026)
. estimates store mcmle
. quietly nwergm flomarriage, edges nodecov(wealth) triangle mainmethod(sa) seed
> (2026)
. estimates store sa
. quietly nwergm flomarriage, edges nodecov(wealth) triangle estimate(cd) seed(2
> 026)
. estimates store cd
. quietly nwergm flomarriage, edges nodecov(wealth) triangle estimate(mple)
. estimates store mple
. estimates table mple mcmle sa cd, b(%8.3f) se(%8.3f)
```

| Variable | mple | mcmle | sa | cd |
|---|---|---|---|---|
| edges | -2.603 | -2.531 | -2.597 | -2.693 |
| | 0.545 | 0.602 | 0.546 | 0.958 |
| nodecov_we~h | 0.010 | 0.010 | 0.011 | 0.013 |
| | 0.005 | 0.005 | 0.005 | 0.010 |
| triangle | 0.036 | -0.074 | -0.106 | -0.161 |
| | 0.447 | 0.543 | 0.620 | 0.996 |

```
Legend: b/se
```

Three further features work as described in Section 4 (the logs are in the supplemental do-file): with `constraints(edges)` the number of ties is fixed, and `edges` is shown as omitted; with two dyads set to missing, `e(N)` is 118 informative dyads and `e(nmissing)` is 2; and a network with tie values is fitted as the binary graph, with one note.

## 7.2 A directed network with covariates: advice among lawyers

The network of a corporate law firm studied by Lazega and van Duijn (1997) has 71 lawyers, 35 associates and 36 partners, in three offices; an arc from $i$ to $j$ means that $i$ names $j$ as a source of advice. The data set is distributed with `nwcommands`; readers who reuse it elsewhere should check the terms of use of the original data. Figure 3 shows the network: with 892 arcs among 71 lawyers it is dense, so the picture is useful for the overall shape (the Boston lawyers form one large cluster, the Hartford and Providence lawyers lie to the side) and not for single ties. Of the 2,485 pairs, 192 are mutual, 508 asymmetric and 1,785 null.

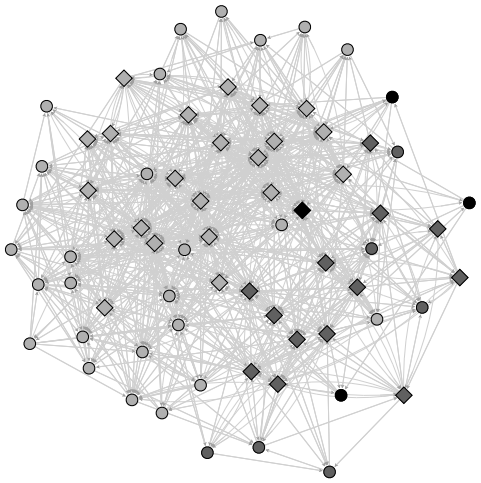


Figure 3: Advice network among 71 lawyers (`nwplot lazega_adv`, do-file `plot_networks.do`). Shading shows the office (light grey Boston, mid grey Hartford, black Providence), diamonds partners and circles associates. Arrowheads are too small to read at this density.

Do lawyers seek advice in their own office and practice, and do partners ask and are they asked more often? A model with these terms is dyad-independent and is fitted by pseudolikelihood:

```
. nwergm lazega_adv, edges nodematch(officen practicen) nodeofactor(statusn) nod
> eifactor(statusn)

Exponential-family random graph model

Network:               =  lazega_adv
Nodes:                 =  71
Ties:                  =  892
Directed:              =  Yes
Estimation:            =  MPLE
```

| lazega_adv | Coefficient | Std. err. | z | P>\|z\| | [95% conf. | interval] |
|---|---|---|---|---|---|---|
| edges | -4.095448 | .1309373 | -31.28 | 0.000 | -4.352081 | -3.838816 |
| nodematch_o~n | 1.422373 | .0887433 | 16.03 | 0.000 | 1.248439 | 1.596306 |
| nodematch_p~n | 1.037945 | .0837376 | 12.40 | 0.000 | .8738223 | 1.202068 |
| nodeofactor~2 | 1.125832 | .0842818 | 13.36 | 0.000 | .9606422 | 1.291021 |
| nodeifactor~2 | .7831546 | .0819723 | 9.55 | 0.000 | .6224919 | .9438172 |

```
Maximum pseudolikelihood estimate (not full ERGM maximum likelihood unless the m
> odel is dyad-independent).

            Null Deviance:    6889.9  on 4970 degrees of freedom
        Residual Deviance:    4009.0  on 4965 degrees of freedom

Log-likelihood: -2004.4927 (df = 5)   AIC:    4019.0   BIC:    4051.5   (MC std.
>  err. =       0)

. estimates store m1
```

The coefficients are conditional log odds. Advice arcs within the same office are more likely by a factor of $\exp(1.42) = 4.1$ in odds than arcs across offices, and within the same practice by $\exp(1.04) = 2.8$, with the other terms held fixed. The factor terms refer to partners (`statusn` 2) against associates (1, the base): partners send advice arcs with $\exp(1.13) = 3.1$ times the odds of associates and receive them with $\exp(0.78) = 2.2$ times the odds. Adding `mutual` (not shown) and a geometrically weighted edgewise shared-partner term, which asks whether the advice arcs close triads beyond what attributes and reciprocity imply, gives a dyad-dependent model fitted by Monte Carlo maximum likelihood with the defaults of `ergm`:

```
. nwergm lazega_adv, edges mutual nodematch(officen practicen) nodeofactor(statu
> sn) nodeifactor(statusn) gwesp(.5) seed(12345)

Exponential-family random graph model

Network:               =  lazega_adv
Nodes:                 =  71
Ties:                  =  892
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  541
```

| lazega_adv | Coefficient | Std. err. | z | P>\|z\| | [95% conf. | interval] |
|---|---|---|---|---|---|---|
| edges | -4.662469 | .1821854 | -25.59 | 0.000 | -5.019546 | -4.305392 |
| mutual | 1.031625 | .1244439 | 8.29 | 0.000 | .7877192 | 1.27553 |
| nodematch_o~n | 1.016895 | .0839039 | 12.12 | 0.000 | .852446 | 1.181343 |
| nodematch_p~n | .7571778 | .0784148 | 9.66 | 0.000 | .6034877 | .910868 |
| nodeofactor~2 | .8242996 | .0907388 | 9.08 | 0.000 | .6464549 | 1.002144 |
| nodeifactor~2 | .4019506 | .0892556 | 4.50 | 0.000 | .2270129 | .5768884 |
| gwesp_.5 | .5597618 | .1091626 | 5.13 | 0.000 | .345807 | .7737166 |

```
MCMLE converged after 5 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

            Null Deviance:    6889.9  on 4970 degrees of freedom
        Residual Deviance:    3908.6  on 4963 degrees of freedom

Log-likelihood: -1954.3217 (df = 7)   AIC:    3922.6   BIC:    3968.2   (MC std.
>  err. =  .10671)

. estimates store m3
```

Reciprocity ($\exp(1.03) = 2.8$ in odds, given the other terms) and closure (0.56, $z = 5.1$) are both clearly positive. The Monte Carlo algorithm needed five iterations; fits of this model took 2 to 3.5 seconds on the machine described in Section 9.3. `estat mcmcdiag` shows whether the simulation behind the estimates mixed. The final sample has 541 draws, the number that the adaptive sample size reached, recorded 4,096 proposals apart (the sampler raised the requested 1,024, as the note under the table below shows); its effective sample size is about 290–360. A fixed sample of 3,000 draws (`mcmleeffectivesize(0)`) has an effective size of about 320–540, at the cost of a longer run.

The diagnostics of the final fit show the same effective sizes and the Geweke statistics: one of seven is flagged at the 5% level, the sender effect of partner status, `nodeofactor(statusn)`, with $z = -2.3$. The Heidelberger–Welch test, which option `heidel` adds, passes for all seven (not shown):

```
. estat mcmcdiag

MCMC diagnostics, final sample                Sample size      =        541
                                              Burn-in          =     16,384
                                              Interval         =      4,096
                                              Acceptance rate  =     0.4998
```

| Statistic | Mean | SD | Autocorr | ESS | Geweke z |
|---|---|---|---|---|---|
| edges | 890.97 | 27.62 | 0.218 | 348.4 | -1.11 |
| mutual | 190.45 | 12.11 | 0.219 | 347.1 | -1.09 |
| nodematch_officen | 671.10 | 23.57 | 0.259 | 319.5 | -0.66 |
| nodematch_practicen | 609.75 | 21.37 | 0.301 | 292.2 | 0.39 |
| nodeofactor_statusn_2 | 616.28 | 20.71 | 0.236 | 335.4 | -2.25* |
| nodeifactor_statusn_2 | 561.06 | 19.25 | 0.231 | 338.7 | -0.76 |
| gwesp_.5 | 1282.16 | 55.17 | 0.207 | 356.3 | -1.09 |

```
The adaptive sampler raised the interval from 1,024 to 4,096.
* |z| > 1.96: the mean of the first 10% of the sample differs from the mean of
  the last 50% at the 5% level (Geweke 1992).

The convergence test of MCMLE was satisfied after 5 iterations. This is
necessary but not sufficient for convergence; option plot draws the trace
plots.
If the ESS is low, raise mcmleeffectivesize(); after mcmleeffectivesize(0),
raise mcmcsamplesize() or mcmcinterval().
```

Figure 4 shows the trace and density of each statistic: a trace that drifts or sticks at one level would be evidence of poor mixing even if the numerical diagnostics looked acceptable.

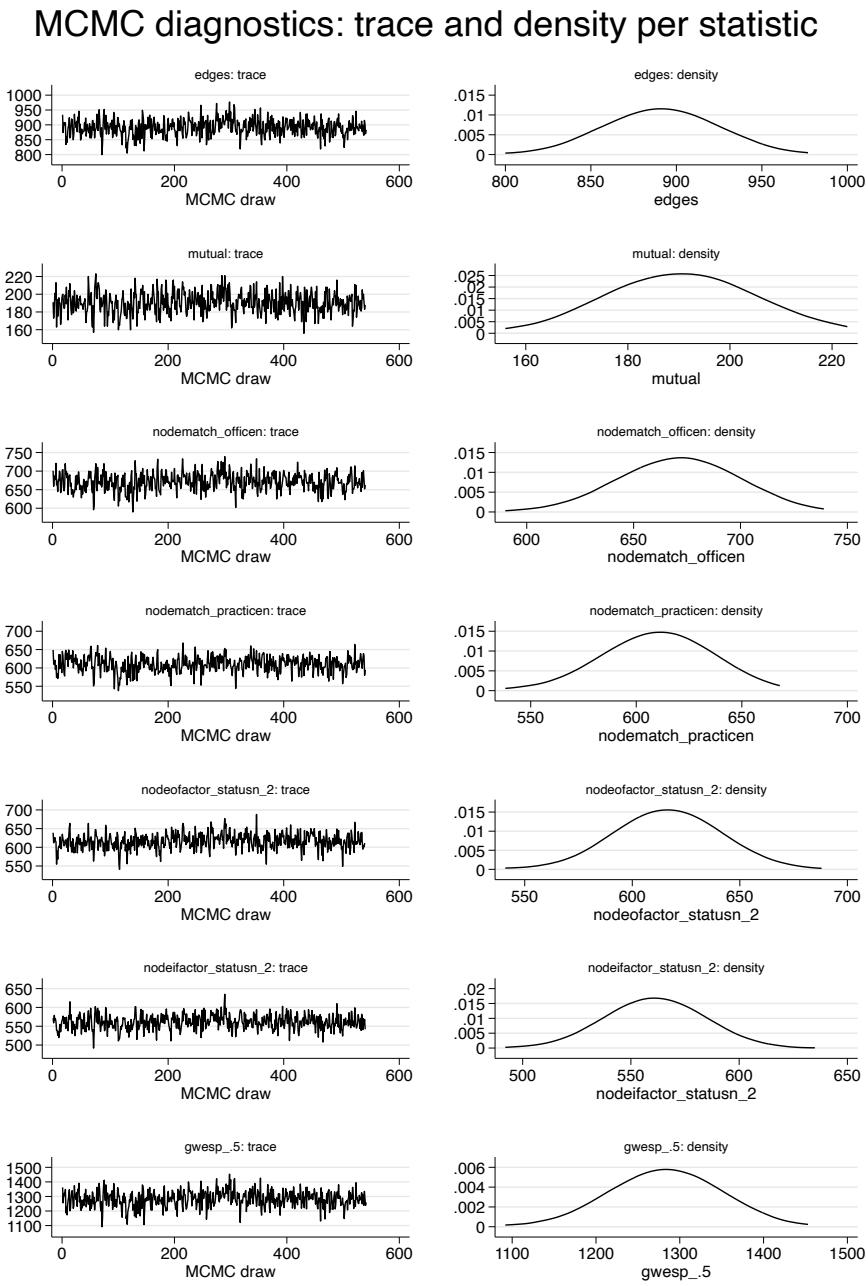



Figure 4: `estat mcmcdiag, plot` after the final advice model: trace (left) and density (right) of each model statistic in the final sample.

Information criteria and the likelihood-ratio test compare the three models (the second adds `mutual`); the closure term improves on reciprocity alone:

```
. estimates stats m1 m2 m3
Akaike´s information criterion and Bayesian information criterion
```

| Model | N | ll(null) | ll(model) | df | AIC | BIC |
|---|---|---|---|---|---|---|
| m1 | 4,970 | -3444.941 | -2004.493 | 5 | 4018.985 | 4051.541 |
| m2 | 4,970 | -3444.941 | -1971.793 | 6 | 3955.585 | 3994.652 |
| m3 | 4,970 | -3444.941 | -1954.322 | 7 | 3922.643 | 3968.222 |

```
Note: BIC uses N = number of observations. See [R] IC note.
. lrtest m2 m3
Likelihood-ratio test
Assumption: m2 nested within m3
 LR chi2(1) =  34.94
Prob > chi2 = 0.0000
```

Goodness of fit asks whether the model reproduces features of the observed network that it was not fitted to. The model statistics are reproduced (by construction, at the estimate) and so are most of the geodesic distances except the pairs at distance 2 and 3, where the model produces too many pairs at distance 2 (3,283 simulated against 3,074 observed) and too few at distance 3 (730 against 991), with Monte Carlo $p$-values of 0.00:

```
. estat gof, gof(model distance) seed(12345)
Goodness of fit: model statistics (100 simulated networks)
```

| Statistic | Observed | Min | Mean | Max | MC p-value |
|---|---|---|---|---|---|
| edges | 892 | 819 | 893.20 | 993 | 0.96 |
| mutual | 192 | 161 | 192.55 | 233 | 0.98 |
| nodematch_officen | 674 | 608 | 679.34 | 752 | 0.80 |
| nodematch_practicen | 610 | 561 | 611.17 | 711 | 0.98 |
| nodeofactor_statusn_2 | 616 | 570 | 615.94 | 667 | 0.92 |
| nodeifactor_statusn_2 | 562 | 516 | 561.23 | 642 | 0.94 |
| gwesp_.5 | 1284.86 | 1134.59 | 1288.29 | 1496.89 | 0.94 |

```
Goodness of fit: minimum geodesic distance (100 simulated networks)
```

| Distance | Observed | Min | Mean | Max | MC p-value |
|---|---|---|---|---|---|
| 1 | 892 | 819 | 893.20 | 993 | 0.96 |
| 2 | 3074 | 3104 | 3282.93 | 3461 | 0.00 |
| 3 | 991 | 527 | 729.97 | 963 | 0.00 |
| 4 | 13 | 0 | 16.97 | 70 | 1.00 |
| 5 | 0 | 0 | 0.14 | 3 | 1.00 |
| Inf | 0 | 0 | 46.79 | 209 | 1.00 |

Figure 5 shows the degree and distance distributions. The observed degree distribution lies mostly inside the simulated ranges, whereas in the distance panel the observed proportion of pairs at distance 2 lies below, and at distance 3 above, the simulated boxes, as the table shows.

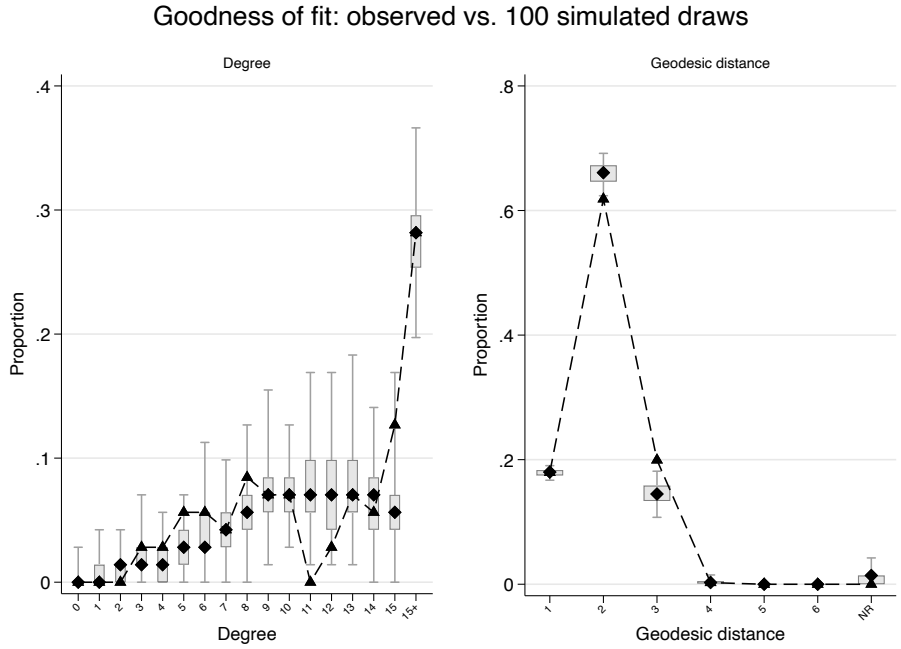



Figure 5: `estat gof, plot` after the final advice model: degree (left) and geodesic-distance (right) distributions, the observed network (dashed line) against 100 simulated networks (boxes and whiskers).

`nwergm simulate, nsim(100) output(stats)` draws networks at the fitted coefficients, and the means of their statistics are those of the goodness-of-fit table.

## 7.3 A two-mode network: the Southern women

The 18 women and 14 social events of the classic data set of Davis et al. (2022) form a two-mode network of 89 attendances (analyzed in Breiger, 1974). With `title` (Miss or Mrs) as the only attribute of the women, the terms of two-mode networks count the nodes of one mode: `bfactor1()` is R's `b1factor`, and `bgwdegree2()` and `bgwdsp1()` are the geometrically weighted degree of the events and the geometrically weighted dyadwise shared partners of the women. Three models with the same attribute and different structural terms were fitted; the listing shows the shared-partner model and the information criteria of all three:

```
. nwergm davis, edges bfactor1(titlen) bgwdsp1(.5) seed(1)
Exponential-family random graph model
Network:              =  davis
Nodes:                =  32
Ties:                 =  89
Directed:             =  No
```

```
Estimation:            =  MCMLE
MCMC sample size:      =  486
```

| davis | Coefficient | Std. err. | z | P>\|z\| | [95% conf. | interval] |
|---|---|---|---|---|---|---|
| edges | -1.176527 | .2987292 | -3.94 | 0.000 | -1.762026 | -.5910288 |
| gwb1dsp_fix~5 | .3024289 | .1292092 | 2.34 | 0.019 | .0491835 | .5556743 |
| b1factor_ti~2 | -.0664987 | .3370617 | -0.20 | 0.844 | -.7271276 | .5941301 |

```
MCMLE converged after 2 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

            Null Deviance:     349.3  on 252 degrees of freedom
        Residual Deviance:     321.9  on 249 degrees of freedom

Log-likelihood:  -160.9260 (df = 3)   AIC:     327.9   BIC:     338.4   (MC std.
>  err. =  .04524)

. estimates store d3

. estimates stats d1 d2 d3

Akaike´s information criterion and Bayesian information criterion
```

| Model | N | ll(null) | ll(model) | df | AIC | BIC |
|---|---|---|---|---|---|---|
| d1 | 252 | -174.6731 | -163.6325 | 2 | 331.2651 | 338.324 |
| d2 | 252 | -174.6731 | -161.9122 | 3 | 329.8244 | 340.4127 |
| d3 | 252 | -174.6731 | -160.926 | 3 | 327.8519 | 338.4402 |

```
Note: BIC uses N = number of observations. See [R] IC note.
```

Whether a woman is a Miss or a Mrs has no effect. Women who attend the same events (shared partners, 0.30 with $z = 2.3$) cluster beyond the density of attendances, and the degrees of the events are more unevenly distributed than a model with only a density term implies (the coefficient of `bgwdegree2` is $-2.2$, $z = -2.6$ in the second model, not shown); the information criteria slightly prefer the shared-partner model. Its goodness of fit adds the degree distributions of both modes:

```
. estat gof, seed(1)

Goodness of fit: model statistics (100 simulated networks)
```

| Statistic | Observed | Min | Mean | Max | MC p-value |
|---|---|---|---|---|---|
| edges | 89 | 78 | 89.74 | 106 | 1.00 |
| gwb1dsp_fixed_.5 | 185.26 | 147.02 | 185.62 | 220.81 | 0.84 |
| b1factor_titlen_2 | 34 | 27 | 34.68 | 45 | 1.00 |

```
Goodness of fit: minimum geodesic distance (100 simulated networks)
```

| Distance | Observed | Min | Mean | Max | MC p-value |
|---|---|---|---|---|---|
| 1 | 89 | 78 | 89.74 | 106 | 1.00 |
| 2 | 205 | 185 | 219.71 | 240 | 0.14 |
| 3 | 163 | 137 | 160.79 | 174 | 0.82 |
| 4 | 39 | 4 | 23.60 | 44 | 0.12 |
| 5 | 0 | 0 | 0.61 | 5 | 1.00 |
| Inf | 0 | 0 | 1.55 | 31 | 1.00 |

```
Goodness of fit: bipartition 1 degree (100 simulated networks)
```

| Degree | Observed | Min | Mean | Max | MC p-value |
|---|---|---|---|---|---|
| 0 | 4 | 4 | 4.15 | 5 | 1.00 |
| 1 | 0 | 0 | 0.48 | 4 | 1.00 |
| 2 | 2 | 0 | 1.20 | 5 | 0.62 |
| 3 | 3 | 0 | 1.97 | 6 | 0.58 |
| 4 | 2 | 0 | 2.37 | 7 | 1.00 |
| 5 | 2 | 0 | 2.32 | 6 | 1.00 |
| 6 | 2 | 0 | 2.09 | 5 | 1.00 |
| 7 | 1 | 0 | 1.52 | 4 | 1.00 |
| 8 | 0 | 0 | 0.90 | 3 | 0.78 |
| 9 | 0 | 0 | 0.62 | 3 | 1.00 |
| 10 | 0 | 0 | 0.24 | 2 | 1.00 |
| 11 | 2 | 0 | 0.11 | 1 | 0.00 |
| 12 | 0 | 0 | 0.02 | 1 | 1.00 |
| 13 | 0 | 0 | 0.01 | 1 | 1.00 |

Goodness of fit: bipartition 2 degree (100 simulated networks)

| Degree | Observed | Min | Mean | Max | MC p-value |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0.01 | 1 | 1.00 |
| 1 | 0 | 0 | 0.03 | 1 | 1.00 |
| 2 | 2 | 0 | 0.36 | 2 | 0.08 |
| 3 | 1 | 0 | 1.60 | 6 | 1.00 |
| 4 | 4 | 0 | 3.45 | 7 | 0.92 |
| 5 | 1 | 0 | 3.81 | 8 | 0.12 |
| 6 | 1 | 0 | 2.63 | 6 | 0.50 |
| 7 | 2 | 0 | 1.45 | 4 | 0.96 |
| 8 | 3 | 0 | 0.48 | 2 | 0.00 |
| 9 | 0 | 0 | 0.16 | 2 | 1.00 |
| 10 | 0 | 0 | 0.02 | 1 | 1.00 |

Goodness of fit: dyadwise shared partner (100 simulated networks)

| DSP | Observed | Min | Mean | Max | MC p-value |
|---|---|---|---|---|---|
| 0 | 91 | 68 | 86.79 | 108 | 0.56 |
| 1 | 40 | 24 | 54.88 | 90 | 0.16 |
| 2 | 50 | 38 | 54.82 | 72 | 0.50 |
| 3 | 32 | 12 | 31.67 | 53 | 1.00 |
| 4 | 17 | 2 | 11.69 | 33 | 0.32 |
| 5 | 7 | 0 | 3.24 | 11 | 0.22 |
| 6 | 5 | 0 | 0.72 | 4 | 0.00 |
| 7 | 1 | 0 | 0.14 | 2 | 0.24 |
| 8 | 1 | 0 | 0.03 | 1 | 0.06 |
| 9 | 0 | 0 | 0.01 | 1 | 1.00 |
| 11 | 0 | 0 | 0.01 | 1 | 1.00 |

The statistics of the model and most counts of the degree and shared-partner distributions are reproduced. Three counts at the upper end are not: the two women who attended 11 events (simulated mean 0.11), the three events with 8 attendees (0.48) and the five dyads with six shared partners (0.72) all have a Monte Carlo $p$-value of 0.00, so the model underproduces the extremes. With 18 women and 14 events the example illustrates the syntax and the output; it is too small for a substantive finding.

## 7.4 Curved terms and operators

A geometrically weighted term with an estimated decay is a curved term. On the friendship network of the students of a school (Faux Mesa High, 205 students, 203 ties, `mesa`), the closure term `gwespfree(.25)` estimates a weight and a decay; the number in parentheses is the starting value. This is a Monte Carlo fit that takes about 2 seconds on the machine used:

```
. nwwebuse mesa, nwclear
. encode race, generate(racen)
. nwergm mesa, edges nodematch(grade racen) gwespfree(.25) seed(1)
Exponential-family random graph model
Network:               =  mesa
Nodes:                 =  205
Ties:                  =  203
Directed:              =  No
Estimation:            =  MCMLE
MCMC sample size:      =  1203
```

| mesa | Coefficient | Std. err. | z | P>\|z\| | [95% conf. | interval] |
|---|---|---|---|---|---|---|
| edges | -6.367347 | .1666168 | -38.22 | 0.000 | -6.69391 | -6.040784 |
| nodematch_g~e | 1.964598 | .1758003 | 11.18 | 0.000 | 1.620036 | 2.30916 |
| nodematch_r~n | .276849 | .1214278 | 2.28 | 0.023 | .0388548 | .5148431 |
| gwesp_weight | 1.35144 | .1369479 | 9.87 | 0.000 | 1.083027 | 1.619853 |
| gwesp_decay | .3820264 | .1056824 | 3.61 | 0.000 | .1748926 | .5891602 |

```
MCMLE converged after 12 iteration(s).
(decay is an estimated curved parameter here - see help nwergm, gw_effects for h
> ow this differs from R ergm´s own curved-model machinery)
```

Homophily in grade is strong, homophily in race is modest, and a clustering weight of 1.35 and a decay of 0.38 describe how fast the effect of an additional shared partner declines. The likelihood and the information criteria are not available after a curved fit by Monte Carlo maximum likelihood.

An operator evaluates a term on a derived network. The advice network is a directed network; on the friendship network of the lawyers, whether friendship triads close among *reciprocated* friendships alone is asked by applying `triangle` to the undirected network in which two lawyers are tied if the friendship is mutual:

```
. nwwebuse lazega, nwclear
. foreach v in office practice status {
  2.     encode `v´, generate(`v´n)
  3. }
. nwergm lazega_fr, edges mutual nodematch(practicen) operators(Symmetrize(trian
> gle, rule(strong))) seed(1)
Exponential-family random graph model
Network:               =  lazega_fr
Nodes:                 =  71
Ties:                  =  575
Directed:              =  Yes
Estimation:            =  MCMLE
MCMC sample size:      =  766
```

```
------------------------------------------------------------------------------
    lazega_fr | Coefficient  Std. err.      z    P>|z|     [95% conf. interval]
--------------+---------------------------------------------------------------
        edges |  -2.736795   .0810857   -33.75   0.000     -2.89572    -2.57787
       mutual |   .9130825   .1676719     5.45   0.000     .5844516    1.241713
nodematch_p~n |   .8967159   .0964408     9.30   0.000     .7076954    1.085736
Symmetrize(~e |   .6922598   .4560459     1.52   0.129    -.2015737    1.586093
------------------------------------------------------------------------------

MCMLE converged after 5 iteration(s).
Evaluating log-likelihood at the estimate (bridge sampling with 16 bridges) ...

            Null Deviance:    6889.9  on 4970 degrees of freedom
        Residual Deviance:    3401.6  on 4966 degrees of freedom

Log-likelihood: -1700.7989 (df = 4)   AIC:    3409.6   BIC:    3435.6   (MC std.
>  err. =  .10562)
```

Closure among mutual friendships is positive but not significant at the 5% level (0.69, $z = 1.5$) on top of reciprocity and same-practice homophily.

## 7.5 Simulating from given coefficients

`nwergm simulate` with a number of nodes draws networks from a fully specified model; the coefficients are given in the order of the terms:

```
. nwergm simulate 16, edges gwesp(.5) theta(-2 .5) seed(42) nsim(3)
. nwsummarize ergmsim_1
```

```
  Network name:  ergmsim_1
  Network id:  4
  Directed: false
  Valued: false
  Two-mode: false
  Nodes: 16
  Selfloop: false
  Edges: 27
  Minimum value:  0
  Maximum value:  1
  Density:  .225
  Temporal: false
```

# 8 Implementation

**Native plugin.** The Metropolis–Hastings sampler, the change statistics of the terms, the observed statistics, the design matrix of the pseudolikelihood and the Newton iterations of curved pseudolikelihood fits are written in C and run in a Stata plugin; `e(native)` records that it ran. The estimating step of the Monte Carlo algorithm, the covariance, the diagnostics and the output are written in Mata and Stata. The plugin is built for macOS, Linux and Windows by continuous integration and installed with the package. The test suite of the package passes on macOS and on Windows; the Linux plugin has not yet been tested on a Linux machine. Its C source is part of the package; if the plugin does not work on a particular computer, `nwinstall, buildplugin(nwergm)` compiles it with the local C compiler and installs it. A Mata implementation of the original terms is kept for platforms without the plugin, for the option `nonative` and for models that exceed the capacity of the plugin. It is much slower (the Florentine model with

`gwesp(.5)` (Section 7.1) took 0.2 seconds in the plugin and 16 seconds in Mata), and `nwergm` prints a warning when it is used. The terms in `structural()`, the constraints and the operators exist in the plugin only.

**Mata library.** The commands share a Mata library (`lnwergm.mlib`), which contains the class definitions of the core library it was compiled against. The package checks at the start of a command that the core and the package libraries match and prints how to reinstall if they do not. Both libraries are built with Stata 17 and work in Stata 17, 18 and 19.

# 9 Comparison with R

## 9.1 How the terms were validated

`nwergm` was developed against `ergm` 4.12.0 (R 4.6.0). For each term, the following were compared with `ergm`: (a) the observed statistic, as `summary(formula)`, on at least two networks (a random network and a real one) of each applicable type, to a tolerance of $10^{-6}$ or better; (b) the maximum pseudolikelihood coefficients and standard errors, as `ergm(estimate="MPLE")`, to $10^{-4}$ in the coefficient; and (c) the Monte Carlo maximum likelihood estimates, over several seeds on each side, which should differ by less than the seed-to-seed spread of `ergm`'s own runs. For the two-mode terms the whole design matrix of change statistics was compared with R's `ergmMPLE()`. Where a term of `ergm` 4.12.0 returns a wrong change statistic, `nwergm` uses the correct one and the statistic still equals R's; this applies to `opentriad`, for which the pseudolikelihood and Monte Carlo estimates therefore differ from those of `ergm` 4.12.0. These checks are repeated by the test suite of the package (about 60 test files), which runs as part of every release. `nwergm version` prints the version of `ergm` used.

## 9.2 Estimates on the same data

Table 2 fits the models of Section 7 (and one small network) in `ergm` 4.12.0 and `nwergm` on the same networks, with the default settings of each program. For the Monte Carlo fits, each program was run with several seeds; the table gives the mean over seeds and the seed-to-seed standard deviation of each program, and the difference between the means in units of the standard error that `ergm` reports.

For the dyad-independent models (the Florentine and the Lazega fits by pseudolikelihood) the programs give the same numbers: the largest difference in a coefficient is $3 \times 10^{-8}$ and in a standard error $10^{-5}$. For the Monte Carlo fits, the means over seeds of the two programs differ by at most 0.09 standard errors of `ergm` for every coefficient in the table, including the operator term of the Lazega friendship model (0.05) and the curved terms (below), and the mean standard errors of the two programs agree within 4%. The seed-to-seed standard deviations of both programs are far smaller than the standard errors of the estimates: for the final advice model, between 0.002 and 0.010 for every coefficient and for both programs, against standard errors of 0.07 to 0.18. A stricter yardstick is the Monte Carlo error of the two means (the seed-to-seed standard deviations divided by the square root of the number of seeds). Of the 23 Monte Carlo

Table 2: Coefficients of `ergm` 4.12.0 and `nwergm` 1.5.4 on the same networks, default settings. Mean over seeds (standard deviation over seeds in parentheses); $\Delta$ is the difference of the means in units of the standard error that `ergm` reports.

| | `ergm` | | `nwergm` | | |
|---|---|---|---|---|---|
| Term | mean | (sd) | mean | (sd) | $\Delta$ |
| *5-node directed network, MCMLE (10 seeds)* | | | | | |
| edges | -0.954 | (0.050) | -0.953 | (0.042) | 0.00 |
| mutual | 0.285 | (0.093) | 0.306 | (0.095) | 0.01 |
| *Florentine marriages, MPLE* | | | | | |
| edges | -2.595 | | -2.595 | | 0.00 |
| nodecov(wealth) | 0.011 | | 0.011 | | 0.00 |
| *Florentine marriages, MCMLE (10 seeds)* | | | | | |
| edges | -2.586 | (0.044) | -2.582 | (0.028) | 0.01 |
| nodecov(wealth) | 0.011 | (0.000) | 0.011 | (0.000) | -0.01 |
| gwesp(.5) | -0.011 | (0.020) | -0.016 | (0.022) | -0.02 |
| *Lazega advice, MPLE* | | | | | |
| edges | -4.095 | | -4.095 | | 0.00 |
| nodematch(office) | 1.422 | | 1.422 | | 0.00 |
| nodematch(practice) | 1.038 | | 1.038 | | 0.00 |
| nodeofactor(status=2) | 1.126 | | 1.126 | | 0.00 |
| nodeifactor(status=2) | 0.783 | | 0.783 | | 0.00 |
| *Lazega advice, MCMLE (5 seeds)* | | | | | |
| edges | -4.648 | (0.007) | -4.663 | (0.008) | -0.08 |
| mutual | 1.034 | (0.010) | 1.025 | (0.004) | -0.07 |
| nodematch(office) | 1.006 | (0.005) | 1.006 | (0.005) | 0.00 |
| nodematch(practice) | 0.759 | (0.006) | 0.756 | (0.004) | -0.04 |
| nodeofactor(status=2) | 0.821 | (0.004) | 0.818 | (0.005) | -0.04 |
| nodeifactor(status=2) | 0.397 | (0.005) | 0.398 | (0.004) | 0.01 |
| gwesp(.5) | 0.557 | (0.003) | 0.566 | (0.002) | 0.09 |
| *Lazega friendship, operator (see text)* | | | | | |
| edges | -2.736 | (0.007) | -2.739 | (0.002) | -0.04 |
| mutual | 0.921 | (0.019) | 0.925 | (0.008) | 0.03 |
| nodematch(practice) | 0.895 | (0.002) | 0.897 | (0.004) | 0.01 |
| Symmetrize(strong)∼triangle | 0.671 | (0.036) | 0.697 | (0.031) | 0.05 |
| *Southern women, two-mode MCMLE (10 seeds)* | | | | | |
| edges | -0.316 | (0.013) | -0.327 | (0.017) | -0.06 |
| b1factor(title=2) | -0.045 | (0.022) | -0.044 | (0.021) | 0.00 |
| gwb2degree(1) | -2.360 | (0.053) | -2.295 | (0.054) | 0.07 |
| *Faux Mesa High, MCMLE (5 seeds)* | | | | | |
| edges | -6.412 | (0.004) | -6.413 | (0.007) | -0.01 |
| nodematch(grade) | 1.991 | (0.005) | 1.988 | (0.007) | -0.02 |
| nodematch(race) | 0.285 | (0.003) | 0.288 | (0.006) | 0.02 |
| gwesp(.25) | 1.452 | (0.007) | 1.450 | (0.003) | -0.02 |
| *Faux Mesa High, curved gwesp (see text)* | | | | | |
| edges | -6.369 | | -6.368 | (0.001) | 0.00 |
| nodematch(grade) | 1.973 | | 1.971 | (0.006) | -0.01 |
| nodematch(race) | 0.279 | | 0.274 | (0.003) | -0.05 |
| gwesp weight | 1.343 | | 1.346 | (0.005) | 0.02 |
| gwesp decay | 0.389 | | 0.388 | (0.005) | -0.02 |

coefficients fitted with several seeds in both programs, 20 differ by less than two such errors, but three do not: `gwesp` and `edges` in the final advice model (5.4 and 3.1 Monte Carlo errors, or 0.010 and 0.014 in the coefficients) and `gwb2degree` in the Southern women model (2.7, or 0.065). These results suggest that small differences between the implementations remain in some models, beyond what the Monte Carlo error alone would lead one to expect. They are at most 0.09 of a standard error of the estimates, so they are unlikely to affect the substantive conclusions drawn from these models. On the five-node network of the first block, which has 6 ties and essentially no information, both programs' means over ten seeds are close relative to the standard errors, which are 0.7 and 1.5.

Two comparisons are not like for like. On the Lazega friendship model, `ergm` stopped with "unconstrained MCMC sampling did not mix at all" for one of five seeds, so the `ergm` column rests on four fits; all five fits of `nwergm` converged. On the curved model for Faux Mesa High, `ergm` stopped for two of three seeds because the number of two-paths on an edge exceeded the cut-off of 30 of its curved shared-partner term (the term's argument `cutoff` can be raised), so the `ergm` column rests on one fit and the three `nwergm` fits all converged. That single `ergm` fit and the mean of the `nwergm` fits differ by 0.05 standard errors or less in every coefficient (0.02 in the weight of the curved term and in its decay); with one `ergm` fit the table cannot show how large the seed-to-seed spread of `ergm` is for this model.

## 9.3 Speed

Table 3 gives the median time of seven fits of each program on identical networks, each with its default control settings; the samplers are not matched beyond that. The machine is an Apple M5 Pro (18 cores, macOS 26.5.1) with R 4.6.0, `ergm` 4.12.0 and Stata/BE 18, and no other Stata or R job ran; the one-minute load average read before each fit was 2.4 to 4.5, of which about one is the fit itself. The two programs were run in turn, one fit at a time (`ergm`, then `nwergm`, then the next seed), so that a slow drift of the machine affects both equally; each fit is a separate process with one untimed warm-up fit, and the time is that of the estimation call alone (`ergm()` in R, the `nwergm` command in Stata). The first four networks are synthetic networks stored in the replication files, the next two are generated with fixed seeds, and the last two are the Lazega and Faux Mesa High fits of Section 7. The undirected networks are loaded as undirected in both programs.

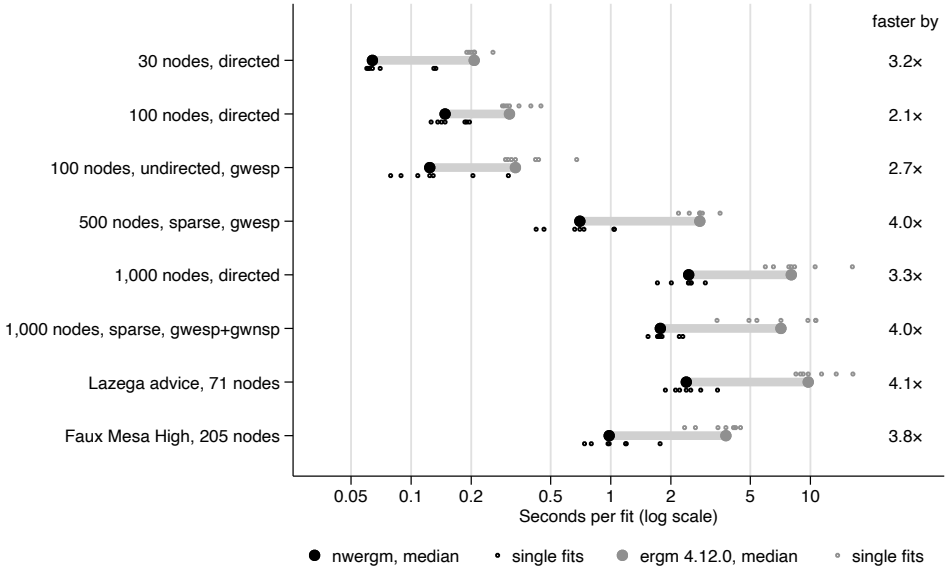



Figure 6: Time of seven fits of each benchmark model (seeds 1 to 7) with `nwergm` (black) and `ergm` 4.12.0 (grey), default settings, log scale. Small circles are single fits, large circles the medians; the number at the right is the ratio of the medians. The numbers are in Table 3.

Table 3: Median time of seven fits (seeds 1 to 7, fitted in turn on the same machine) of `ergm` 4.12.0 and of `nwergm` on identical networks, default settings; ratio of the medians, `ergm` divided by `nwergm`; Monte Carlo iterations in the seven fits of each program (minimum to maximum).

| Benchmark | `ergm` (s) | `nwergm` (s) | ratio | `ergm` it. | `nwergm` it. |
|---|---|---|---|---|---|
| 30 nodes, directed: edges, mutual | 0.21 | 0.06 | 3.2 | 1 | 1 |
| 100 nodes, directed: + nodematch | 0.31 | 0.15 | 2.1 | 1–3 | 1–4 |
| 100 nodes, undirected: edges, gwesp | 0.33 | 0.12 | 2.7 | 1 | 1 |
| 500 nodes, sparse, undir.: + nodematch, gwesp | 2.8 | 0.70 | 4.0 | 3–5 | 2–6 |
| 1000 nodes, directed: edges, mutual, nodematch | 8.0 | 2.5 | 3.3 | 2–3 | 1–3 |
| 1000 nodes, sparse, undir.: edges, gwesp, gwnsp | 7.1 | 1.8 | 4.0 | 2–6 | 2–4 |
| Lazega advice, 71 nodes: seven terms | 9.8 | 2.4 | 4.1 | 6–8 | 6–9 |
| Faux Mesa High, 205 nodes: gwesp(.25) | 3.8 | 0.98 | 3.8 | 5–11 | 6–10 |

With the defaults of both programs, `nwergm` was faster than `ergm` on all eight models (Figure 6), by factors of 2.1 (the 100-node directed network) to 4.1 (the Lazega model); the two fits of 1000 nodes take 2.5 and 1.8 seconds against 8.0 and 7.1 seconds, and the fits of the small networks 0.06 to 0.15 seconds against 0.21 to 0.33. Every call has a fixed overhead of the order of 0.05 to 0.25 seconds (parsing, set-up, returning the results),

which is much of the time of the small models.

The measurements suggest that the speed does not come from sampling less. Both programs run the same algorithm with the same defaults: the proposal SPDyad, the adaptive sample size with the effective size computed from the multivariate spectral density of the estimating equations, the same convergence test and 16 bridges for the log-likelihood (Section 5). They also iterate alike (1 to 6 iterations on the first six models and 6 to 9 on the Lazega model; `ergm`: 1 to 6 and 6 to 8). I counted the Metropolis–Hastings proposals of the fits with seeds 1 to 3 (with a counter added to a copy of each program, because neither reports them), including burn-in and the bridges of the log-likelihood: `nwergm` made 0.7 to 1.5 times as many as `ergm` on the eight models (9.9 against 9.1 million on the Lazega model), and the differences follow the numbers of iterations, which vary with the seed in both programs. The gain is in the cost of a proposal and of the work around the sampler: dividing the time of these fits by their proposals gives 2.2 to 4.6 times less time per proposal for `nwergm`, and with a fixed workload (one iteration with 4,000 draws at interval 1,024, no adaptive sample size) a proposal of `nwergm` cost 0.08 to 0.25 microseconds and one of `ergm` 0.20 to 1.04. The times of both programs vary with the seed, because the number of iterations does (for example the sparse 1000-node model took 1.5 to 2.3 seconds in `nwergm` and 3.4 to 10.7 seconds in `ergm`), and the order of the programs on a single fit could differ from the order of the medians; the ratios are those of the medians. The estimates of these fits are those compared with `ergm`'s in Table 2.

**Models with one coefficient per node.** `sender` and `receiver`, and factor terms on attributes with many levels, give models with hundreds or thousands of coefficients. Without a dyad-dependent term such a model is fitted by the pseudolikelihood, which is then the likelihood, with a sparse design: `edges sender receiver` took 1.0 second on a random directed network of 400 nodes (799 coefficients; `ergm` 573 seconds, largest difference of the estimates $1.4 \times 10^{-6}$) and about 8 seconds on 1,000 nodes (1,999 coefficients, Stata/BE). With a dyad-dependent term the work of an iteration grows with the square of the number of coefficients, because both programs draw at least 32 networks per coefficient, so such fits take much longer than fits of the same network without the per-node terms.

The steps after a fit can take longer than the fit on large networks: on a random network of 1,000 nodes with `edges` and `gwesp`, the fit took 1 second, `estat gof` 39 seconds (the triad census by the algorithm of Batagelj and Mrvar (2001), distances by breadth-first search) and 100 simulated networks 28 seconds (single runs).

The timings were made on 9 November 2026, on one machine, with Stata 18 on macOS; Stata 17, Linux and Windows were not timed. They hold for the eight models of the table: the speed relative to `ergm` depends on the model and the data, and a model that runs in the Mata sampler instead of the plugin (Section 8) is much slower than `ergm`.

## 10 What is not implemented

Of the 139 terms of `ergm` 4.12.0, 14 are not implemented: the 13 terms for valued networks (`atleast`, `atmost`, `cdf`, `cyclicalweights`, `equalto`, `greaterthan`, `ininterval`, `nodecovar`, `nodeicovar`, `nodeocovar`, `smallerthan`, `sum`, `transitiveweights`) and the operator `B.` `nwergm` models the presence of ties; a valued network is fitted as the binary graph (Section 4), and the values are not an outcome. Two terms are implemented in part: `Curve()` for linear maps of the coefficients (`map(rep)` and `map(matrix())`; not `minpar()`, `maxpar()` or `cov()`), and `altkstar` with an estimated decay, which is available by pseudolikelihood with starting values from `init()` (`ergm` itself cannot fit it by Monte Carlo maximum likelihood on the test networks). Further limits are: the two-sided form of `attrs` in `S()`, dyadic covariates inside `S()` and `Project()`, and curved terms inside an operator; the constraints `hamming`, `ChangeStats()`, `Dyads()` and `I()`; missing dyads together with the constraints `freedyads()`, `blockdiag()` or `fixdensity`; directed two-mode networks; and the packages built on `ergm`, `tergm` for dynamic networks, `ergm.count` for count data and `ergm.multi` for several networks. `ergm` remains the reference implementation and the more complete program: where it is available it is the right tool for models that `nwergm` lacks, and the comparison above can be repeated on the user's data. The manual (Grund, 2026c) lists the status of every term of `ergm`.

## 11 Conclusions

`nwergm` brings exponential-family random graph models to Stata: 123 of the 139 terms of `ergm` 4.12.0 fully and two in part, four estimators with the defaults of `ergm`, constraints, missing-data handling, diagnostics, goodness of fit, information criteria, prediction and simulation, in a Monte Carlo sampler that runs as compiled code. On the same data, the estimates agree with those of `ergm` to within a tenth of a standard error. With the defaults of `ergm`, including its adaptive Monte Carlo sample size, the current version fits the eight benchmark models 2.1 to 4.1 times faster than `ergm`, with about as many Monte Carlo proposals. Because an ERGM is fitted by simulation, an analysis should report the effective sample size, look at the trace plots and check the goodness of fit on features the model was not fitted to.

## Also see

`nwcommands` (Grund, 2026a), which includes `nwplot` for plotting observed and simulated networks; further model packages and extensions: `https://nwcommands.org`.


## Acknowledgments

`nwergm` would not exist without the statnet project. The `ergm` package (Hunter and Handcock, 2006; Handcock et al., 2008), developed by the statnet team and many contributors, is the reference implementation that `nwergm` follows and is compared with; its source code and documentation were the specification for the Stata code. I am grateful to the developers for making their software public.

## Use of AI in developing the software

The software was developed with the help of an AI coding assistant (Claude Code, Anthropic), working under my direction. I specified the scope, reviewed the code, and am responsible for the package.

## About the author

Thomas U. Grund is a sociologist at the Institute of Sociology, RWTH Aachen University, working on social network analysis and computational social science. He is the author of `nwcommands`, a Stata package for the analysis of network data, and of `nwergm`, which builds on it.

# 12 References


Batagelj, V., and A. Mrvar. 2001. A subquadratic triad census algorithm for large sparse networks with small maximum degree. *Social Networks* 23(3): 237–243.

Breiger, R. L. 1974. The Duality of Persons and Groups. *Social Forces* 53(2): 181.

Davis, A., B. B. Gardner, and M. R. Gardner. 2022. *Deep South.* University of Chicago Press. Reissue of the 1941 edition, with a foreword by Isabel Wilkerson.

van Duijn, M. A. J., K. J. Gile, and M. S. Handcock. 2009. A Framework for the Comparison of Maximum Pseudo-Likelihood and Maximum Likelihood Estimation of Exponential Family Random Graph Models. *Social Networks* 31(1): 52–62.

Firth, D. 1993. Bias Reduction of Maximum Likelihood Estimates. *Biometrika* 80(1): 27–38.

Geyer, C. J., and E. A. Thompson. 1992. Constrained Monte Carlo Maximum Likelihood for Dependent Data. *Journal of the Royal Statistical Society, Series B* 54(3): 657–683.

Grund, T. U. 2026a. nwcommands: Scalable and Extensible Network Analysis in Stata. Manuscript submitted to the Stata Journal.

———. 2026b. nwplot: Flexible, Publication-Quality Network Visualization in Stata. Manuscript submitted to the Stata Journal.

———. 2026c. nwergm: Exponential Random Graph Models in Stata. Manual. Zenodo. `https://doi.org/10.5281/zenodo.23661331`.

Handcock, M. S., and K. J. Gile. 2010. Modeling Social Networks from Sampled Data. *The Annals of Applied Statistics* 4(1): 5–25.

Handcock, M. S., D. R. Hunter, C. T. Butts, S. M. Goodreau, and M. Morris. 2008. statnet: Software Tools for the Representation, Visualization, Analysis and Simulation of Network Data. *Journal of Statistical Software* 24(1): 1–11.

Hummel, R. M., D. R. Hunter, and M. S. Handcock. 2012. Improving Simulation-Based Algorithms for Fitting ERGMs. *Journal of Computational and Graphical Statistics* 21(4): 920–939.

Hunter, D. R. 2007. Curved Exponential Family Models for Social Networks. *Social Networks* 29(2): 216–230.

Hunter, D. R., S. M. Goodreau, and M. S. Handcock. 2008a. Goodness of Fit of Social Network Models. *Journal of the American Statistical Association* 103(481): 248–258.

Hunter, D. R., and M. S. Handcock. 2006. Inference in Curved Exponential Family Models for Networks. *Journal of Computational and Graphical Statistics* 15(3): 565–583.

Hunter, D. R., M. S. Handcock, C. T. Butts, S. M. Goodreau, and M. Morris. 2008b. ergm: A Package to Fit, Simulate and Diagnose Exponential-Family Models for Networks. *Journal of Statistical Software* 24(3).

Krivitsky, P. N. 2017. Using Contrastive Divergence to Seed Monte Carlo MLE for Exponential-Family Random Graph Models. *Computational Statistics & Data Analysis* 107: 149–161.

Krivitsky, P. N., D. R. Hunter, M. Morris, and C. Klumb. 2023. ergm 4: New Features for Analyzing Exponential-Family Random Graph Models. *Journal of Statistical Software* 105(6).

Lazega, E., and M. van Duijn. 1997. Position in Formal Structure, Personal Characteristics and Choices of Advisors in a Law Firm: A Logistic Regression Model for Dyadic Network Data. *Social Networks* 19(4): 375–397.

Lusher, D., J. Koskinen, and G. Robins, eds. 2013. *Exponential Random Graph Models for Social Networks: Theory, Methods, and Applications*. New York: Cambridge University Press.

Morris, M., M. S. Handcock, and D. R. Hunter. 2008. Specification of Exponential-Family Random Graph Models: Terms and Computational Aspects. *Journal of Statistical Software* 24(4): 1–24.

Padgett, J. F., and C. K. Ansell. 1993. Robust Action and the Rise of the Medici, 1400–1434. *American Journal of Sociology* 98(6): 1259–1319.

Snijders, T. A. B. 2002. Markov chain Monte Carlo estimation of exponential random graph models. *Journal of Social Structure* 3(2): 1–40.

Wang, P., G. Robins, and P. Pattison. 2009. *PNet: Program for the Simulation and Estimation of Exponential Random Graph ($p^*$) Models*. Melbourne School of Psychological Sciences, University of Melbourne.